TL08 Planificadores

Índice

1 Introducción

Descenso por gradiente estocástico (SGD): dado un $\boldsymbol{\theta}_0$, minimiza la pérdida $\mathcal{L}$ iterativamente, moviéndose en la dirección de descenso más pronunciada, esto es, la dada por el neg-gradiente de la pérdida calculada en cada paso (batch)

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \, \boldsymbol{g}(\boldsymbol{\theta}_t) \quad \text{donde} \quad \eta_t > 0 \qquad (t = 0, 1, \ldots)$$

Planificador: función heurística para determinar el learning rate η_t en función de t (número de batches procesados)

Factor de aprendizaje constante: SGD puede no converger o hacerlo muy lentamente

$$\eta_t = \eta$$

Caída escalonada: con **decay rate** γ y **decay step** s

$$\eta_t = \eta_0 \, \gamma^{\lfloor t/s \rfloor}$$

Ejemplo: con $\eta_0 = 0.1$, $\gamma = 0.5$ y $s = 2$

$$\eta_0 = 0.1 \qquad \eta_1 = 0.1 \cdot 0.5^{\lfloor 1/2 \rfloor} = 0.1 \qquad \eta_2 = 0.1 \cdot 0.5^{\lfloor 2/2 \rfloor} = 0.05 \qquad \cdots$$

Objetivo: familiarizarse con algunos planficadores populares

2 Planificadores

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
import torch; import torch.optim as optim; import torch.nn as nn
```

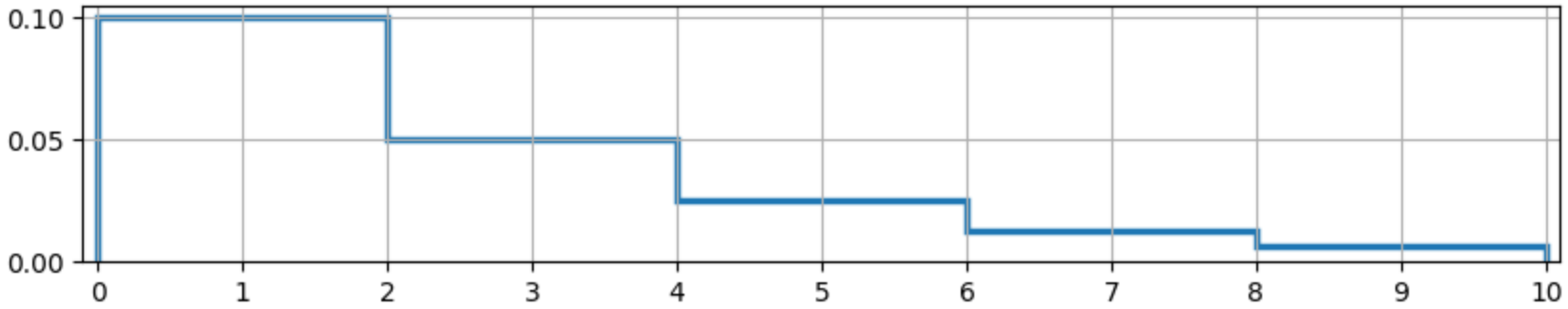
In [ ]:
```
def get_lrs(optimizer, scheduler, steps=100, val_loss=None):
    lrs = []
    for t in range(steps):
        lrs += scheduler.get_last_lr(); optimizer.step()
        if val_loss == None: scheduler.step()
        else: scheduler.step(val_loss[t])
    return lrs
```

Clase base: `LRScheduler((optimizer, last_epoch=-1)`

- `optimizer`: optimizador cuyo learning rate se ajusta
- `last_epoch=-1`: índice de la última época vista por el planificador

Caída escalonada: `StepLR(optimizer, step_size, gamma=0.1, last_epoch=-1)`

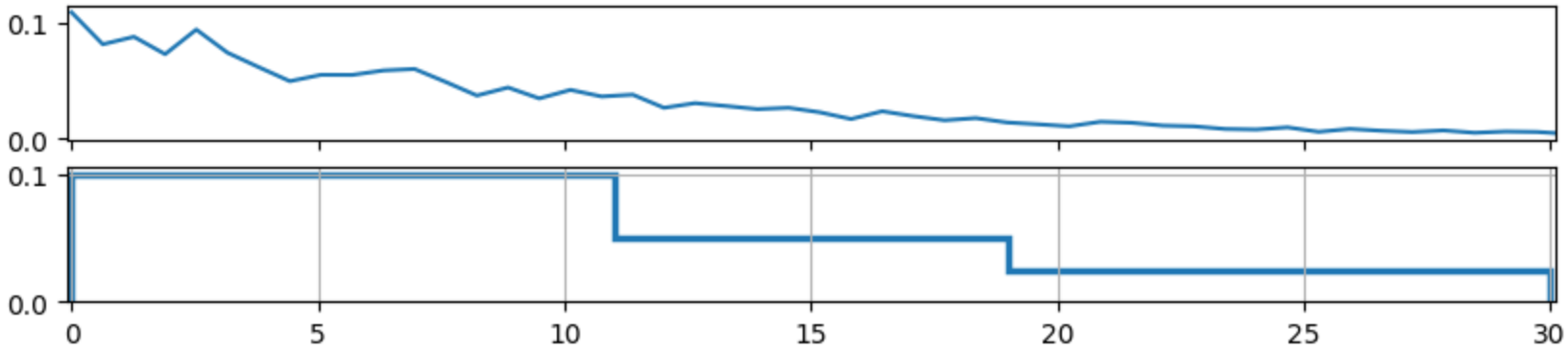
In [ ]:
```
model = nn.Linear(10, 2); optimizer = optim.SGD(model.parameters(), lr=0.1)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=2, gamma=0.5)
steps = 10; lrs = get_lrs(optimizer, scheduler, steps=steps)
_, ax = plt.subplots(figsize=(10, 1.75)); ax.stairs(lrs, linewidth=2.5); ax.grid()
ax.set(xlim=(-.1, steps+.1), xticks=np.arange(0, steps+1));
```



ReduceLROnPlateau (RLROP): `ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=10, threshold=0.0001, threshold_mode='rel', cooldown=0, min_lr=0, eps=1e-08)`

- `mode='min'`: el lr se reduce cuando la métrica monitorizada deja de decrecer (`min`) o crecer (`max`)
- `factor=0.1`: factor de reducción del lr
- `patience=10`: número de pasos sin mejora tras los cuales se reduce el lr
- `threshold=0.0001`: umbral mínimo para considerar que una mejora es significativa
- `threshold_mode='rel'`: umbral relativo al mejor valor hallado (`rel`) o absoluto (`abs`)
- `cooldown=0`: número de pasos a esperar para seguir con el proceso tras una reducción del lr
- `min_lr=0.0`: cota inferior para el lr
- `eps=1e-08`: si la diferencia entre el nuevo lr y el anterior es menor que eps, el lr no se actualiza

In [ ]:
```
model = nn.Linear(10, 2); optimizer = optim.SGD(model.parameters(), lr=0.1)
steps = 30; x = np.linspace(0, steps+1); exp_lambda = .1
val_loss = torch.from_numpy(exp_lambda * np.exp(-exp_lambda * x))
val_loss += val_loss * ( torch.rand_like(val_loss, ) - .5 ) * .5
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, factor=.5, patience=2)
lrs = get_lrs(optimizer, scheduler, steps=steps, val_loss=val_loss)
_, (ax_loss, ax) = plt.subplots(2, figsize=(10, 2), sharex=True)
ax_loss.plot(x, val_loss); ax.stairs(lrs, linewidth=2.5); ax.grid()
ax.set(xlim=(-.1, steps+.1), xticks=np.arange(0, steps+1, 5));
```

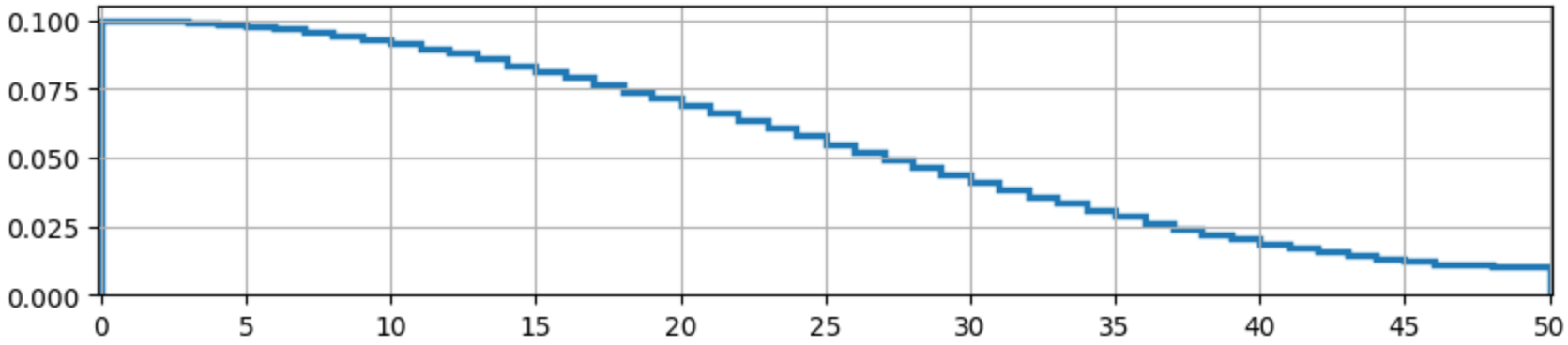


CosineAnnealing: `CosineAnnealingLR(optimizer, T_max, eta_min=0.0, last_epoch=-1)`

$$\eta_{t+1} = \eta_{\min} + (\eta_t - \eta_{\min}) \cdot \frac{1 + \cos\left(\frac{(T_{cur}+1)\,\pi}{T_{max}}\right)}{1 + \cos\left(\frac{T_{cur}\,\pi}{T_{max}}\right)}$$

- T_{cur}: número de pasos desde el inicio de la época actual
- T_{max}: máximo número de pasos en una época
- `T_max`: máximo número de pasos
- `eta_min=0.`: learning rate mínimo

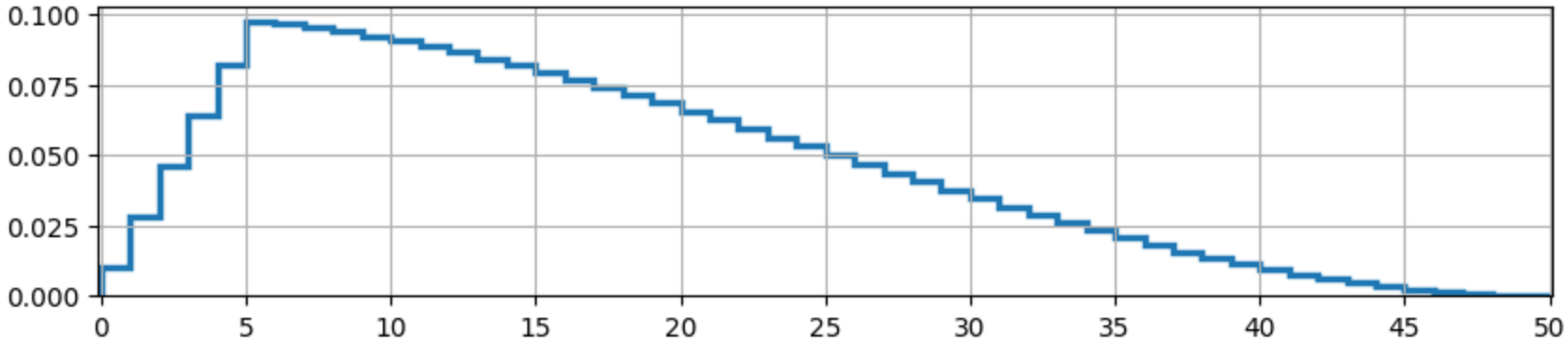
In [ ]:
```
model = nn.Linear(10, 2); optimizer = optim.SGD(model.parameters(), lr=0.1)
steps = 50; scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, steps, eta_min=.010)
lrs = get_lrs(optimizer, scheduler, steps=steps)
_, ax = plt.subplots(figsize=(10, 2)); ax.stairs(lrs, linewidth=2.5); ax.grid()
ax.set(xlim=(-.1, steps+.1), xticks=np.arange(0, steps+1, 5));
```



CosineAnnealing with linear warmup: `CosineLRScheduler( optimizer, t_initial, lr_min=0.0, cycle_mul=1.0, cycle_decay=1.0, cycle_limit=1, warmup_t=0, warmup_lr_init=0.0, warmup_prefix=False, t_in_epochs=True, noise_range_t=None, noise_pct=0.67, noise_std=1.0, noise_seed=42, k_decay=1.0, initialize=True)`

- `t_initial`: total de pasos
- `warmup_t=0`: pasos de warmup lineal inicial

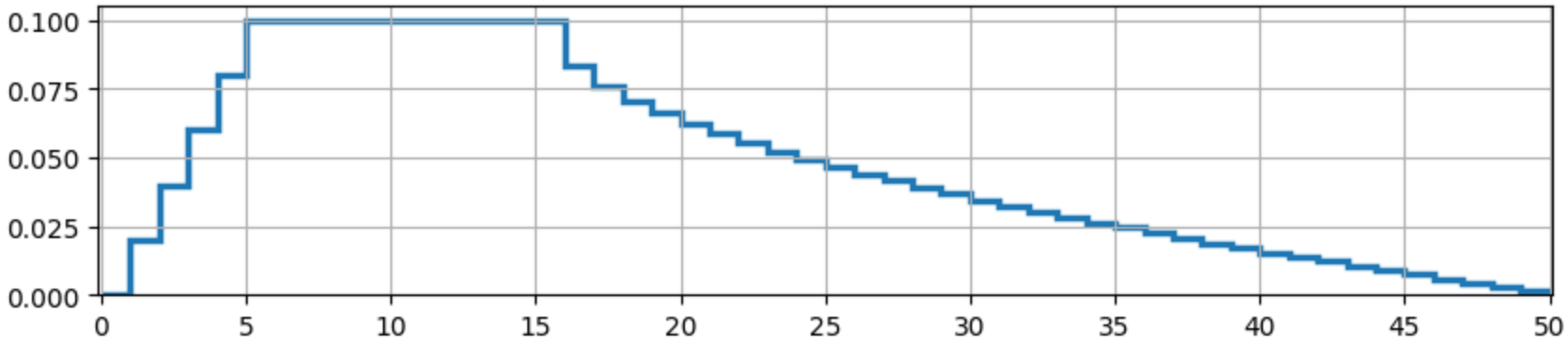
In [ ]:
```
from timm.scheduler.cosine_lr import CosineLRScheduler
model = nn.Linear(10, 2); optimizer = optim.SGD(model.parameters(), lr=0.1)
steps = 50; scheduler = CosineLRScheduler(optimizer, t_initial=steps, warmup_t=5, warmup_lr_init=1e-2)
lrs = [scheduler._get_lr(t)[0] for t in range(steps)]
_, ax = plt.subplots(figsize=(10, 2)); ax.stairs(lrs, linewidth=2.5); ax.grid()
ax.set(xlim=(-.1, steps+.1), xticks=np.arange(0, steps+1, 5));
```



Warmup-Stable-Decay (WSD): `get_wsd_schedule(optimizer, num_warmup_steps, num_stable_steps, num_decay_steps, min_lr_ratio=0.0, num_cycles=0.5, cooldown_type='1-sqrt', last_epoch=-1)`

- `num_warmup_steps`: pasos de warmup lineal inicial
- `num_stable_steps`: pasos de stable intermedio
- `num_decay_steps`: pasos de decay final

In [ ]:
```
from pytorch_optimizer import get_wsd_schedule
model = nn.Linear(10, 2); optimizer = optim.SGD(model.parameters(), lr=0.1)
steps = 50; scheduler = get_wsd_schedule(optimizer, 5, 10, 35)
lrs = get_lrs(optimizer, scheduler, steps=steps)
_, ax = plt.subplots(figsize=(10, 2)); ax.stairs(lrs, linewidth=2.5); ax.grid()
ax.set(xlim=(-.1, steps+.1), xticks=np.arange(0, steps+1, 5));
```



3 Ejemplo

`exp.py`: rutina experimental para schedulers no guiados por una métrica

In [ ]:
```python
import torch

def exp(model, device, train_loader, test_loader, optimizer, scheduler, epochs=15):
    loss_fn = torch.nn.CrossEntropyLoss()
    for epoch in range(epochs):
        print(f'Epoch {epoch}: train:', end=' ')
        model.train(); trsize, trbatches, trloss, tracc = 0, 0, 0, 0
        for batch in train_loader:
            X, y = batch['X'].to(device), batch['y'].to(device); trsize += len(X); trbatches += 1
            pred = model(X); loss = loss_fn(pred, y); trloss += loss.item()
            tracc += (pred.argmax(1) == y).type(torch.float).sum().item()
            loss.backward(); optimizer.step(); optimizer.zero_grad(); scheduler.step()
        trloss /= trbatches; tracc /= trsize
        print(f'loss {trloss:g} acc {tracc:.2%} test:', end=' ')
        model.eval(); tesize, tebatches, teloss, teacc = 0, 0, 0, 0
        with torch.no_grad():
            for batch in test_loader:
                X, y = batch['X'].to(device), batch['y'].to(device); tesize += len(X); tebatches += 1
                pred = model(X); teloss += loss_fn(pred, y).item()
                teacc += (pred.argmax(1) == y).type(torch.float).sum().item()
        teloss /= tebatches; teacc /= tesize
        print(f'loss {teloss:g} acc {teacc:.2%}')
```

Ejemplo: CIFAR-10 con CosineAnnealing

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; import torch; import timm; from exp import exp
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10).to(device)
```

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("uoft-cs/cifar10").with_format("torch").rename_columns({'img': 'X', 'label': 'y'})
def datanorm(example):
    example['X'] = example['X'].to(torch.float) / 255.
    return example
ds = ds.map(datanorm, batched=True)
train_ds = ds['train'].to_iterable_dataset(num_shards=1024)
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)
train_loader = DataLoader(train_ds, batch_size=32)
test_loader = DataLoader(test_ds, batch_size=32)
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)
steps = 20; scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, steps, eta_min=1e-5)
exp(model, device, train_loader, test_loader, optimizer, scheduler, epochs=steps)
```

```
Epoch 0: train: loss 2.30813 acc 27.70% test: loss 2.19528 acc 21.78%
Epoch 1: train: loss 1.84899 acc 32.92% test: loss 1.62432 acc 40.43%
Epoch 2: train: loss 1.4318 acc 48.32% test: loss 1.34952 acc 52.35%
Epoch 3: train: loss 1.16571 acc 59.06% test: loss 1.07064 acc 62.46%
Epoch 4: train: loss 0.957133 acc 67.01% test: loss 0.895093 acc 68.93%
Epoch 5: train: loss 0.821895 acc 71.74% test: loss 0.825412 acc 71.95%
Epoch 6: train: loss 0.694765 acc 76.39% test: loss 0.812678 acc 73.16%
Epoch 7: train: loss 0.593442 acc 79.78% test: loss 0.727295 acc 75.98%
Epoch 8: train: loss 0.504465 acc 82.91% test: loss 0.691837 acc 77.00%
Epoch 9: train: loss 0.429899 acc 85.47% test: loss 0.740643 acc 77.30%
Epoch 10: train: loss 0.374486 acc 87.31% test: loss 0.796002 acc 76.24%
Epoch 11: train: loss 0.332235 acc 88.74% test: loss 1.1227 acc 69.85%
Epoch 12: train: loss 0.280527 acc 90.64% test: loss 0.774245 acc 78.04%
Epoch 13: train: loss 0.240291 acc 91.99% test: loss 1.35432 acc 67.31%
Epoch 14: train: loss 0.222081 acc 92.56% test: loss 1.20723 acc 69.51%
Epoch 15: train: loss 0.190548 acc 93.55% test: loss 0.842597 acc 78.26%
Epoch 16: train: loss 0.173741 acc 94.17% test: loss 0.904766 acc 78.27%
Epoch 17: train: loss 0.157968 acc 94.53% test: loss 0.895832 acc 79.06%
Epoch 18: train: loss 0.156604 acc 94.71% test: loss 0.911176 acc 77.79%
Epoch 19: train: loss 0.130545 acc 95.53% test: loss 1.01241 acc 78.21%
```

4 Ejercicio

Ejercicio: prueba algún planificador con MNIST y FashionMNIST

MNIST:

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; import torch; import timm; from exp import exp
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10, in_chans=1).to(device)
```

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("ylecun/mnist").with_format("torch").rename_columns({'image': 'X', 'label': 'y'})
def datanorm(example):
    example['X'] = example['X'].to(torch.float) / 255.
    return example
ds = ds.map(datanorm, batched=True)
train_ds = ds['train'].to_iterable_dataset(num_shards=1024)
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)
train_loader = DataLoader(train_ds, batch_size=32)
test_loader = DataLoader(test_ds, batch_size=32)
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)
steps = 20; scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, steps, eta_min=1e-5)
exp(model, device, train_loader, test_loader, optimizer, scheduler, epochs=steps)
```

```
Epoch 0: train: loss 0.360347 acc 92.36% test: loss 0.0578652 acc 98.31%
Epoch 1: train: loss 0.139882 acc 97.07% test: loss 0.0431343 acc 98.71%
Epoch 2: train: loss 0.0694366 acc 98.08% test: loss 0.0466572 acc 98.64%
Epoch 3: train: loss 0.0764315 acc 98.04% test: loss 0.0468985 acc 98.60%
Epoch 4: train: loss 0.0817561 acc 98.07% test: loss 0.108234 acc 96.68%
Epoch 5: train: loss 0.0502064 acc 98.64% test: loss 0.0399951 acc 98.87%
Epoch 6: train: loss 0.0477143 acc 98.68% test: loss 0.04147 acc 98.93%
Epoch 7: train: loss 0.0359118 acc 99.04% test: loss 0.0452152 acc 98.55%
Epoch 8: train: loss 0.0383849 acc 98.99% test: loss 0.0368849 acc 99.03%
Epoch 9: train: loss 0.023773 acc 99.31% test: loss 0.0416461 acc 98.89%
Epoch 10: train: loss 0.0241647 acc 99.31% test: loss 0.0621326 acc 98.58%
Epoch 11: train: loss 0.033237 acc 99.16% test: loss 0.0279418 acc 99.22%
Epoch 12: train: loss 0.0189275 acc 99.43% test: loss 0.0336875 acc 98.98%
Epoch 13: train: loss 0.0348451 acc 99.19% test: loss 0.0321667 acc 99.05%
Epoch 14: train: loss 0.0260008 acc 99.34% test: loss 0.0302305 acc 99.21%
Epoch 15: train: loss 0.0159016 acc 99.55% test: loss 0.0278396 acc 99.32%
Epoch 16: train: loss 0.0235641 acc 99.40% test: loss 0.0361614 acc 99.01%
Epoch 17: train: loss 0.0117205 acc 99.67% test: loss 0.0434699 acc 98.98%
Epoch 18: train: loss 0.014986 acc 99.57% test: loss 0.0366191 acc 99.01%
Epoch 19: train: loss 0.0133863 acc 99.62% test: loss 0.0490546 acc 98.91%
```

Fashion-MNIST:

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; import torch; import timm; from exp import exp
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10, in_chans=1).to(device)
```

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("zalando-datasets/fashion_mnist").with_format("torch")
ds = ds.rename_columns({'image': 'X', 'label': 'y'})
def datanorm(example):
    example['X'] = example['X'].to(torch.float) / 255.
    return example
ds = ds.map(datanorm, batched=True)
train_ds = ds['train'].to_iterable_dataset(num_shards=1024)
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)
train_loader = DataLoader(train_ds, batch_size=32)
test_loader = DataLoader(test_ds, batch_size=32)
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)
steps = 20; scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, steps, eta_min=1e-5)
exp(model, device, train_loader, test_loader, optimizer, scheduler, epochs=steps)
```

```
Epoch 0: train: loss 0.782196 acc 77.35% test: loss 0.497043 acc 81.79%
Epoch 1: train: loss 0.55586 acc 82.06% test: loss 0.392175 acc 85.32%
Epoch 2: train: loss 0.480537 acc 84.58% test: loss 0.35894 acc 86.65%
Epoch 3: train: loss 0.338483 acc 88.06% test: loss 0.330318 acc 88.04%
Epoch 4: train: loss 0.340302 acc 88.36% test: loss 0.346436 acc 88.02%
Epoch 5: train: loss 0.276466 acc 89.97% test: loss 0.365552 acc 86.85%
Epoch 6: train: loss 0.31308 acc 89.54% test: loss 0.287404 acc 89.81%
Epoch 7: train: loss 0.253936 acc 91.11% test: loss 0.283611 acc 89.92%
Epoch 8: train: loss 0.227574 acc 91.84% test: loss 0.28435 acc 89.71%
Epoch 9: train: loss 0.21861 acc 92.30% test: loss 0.253178 acc 91.16%
Epoch 10: train: loss 0.188243 acc 93.12% test: loss 0.276807 acc 90.39%
Epoch 11: train: loss 0.176133 acc 93.60% test: loss 0.266306 acc 91.04%
Epoch 12: train: loss 0.190565 acc 93.23% test: loss 0.284002 acc 90.76%
Epoch 13: train: loss 0.149726 acc 94.42% test: loss 0.308228 acc 90.45%
Epoch 14: train: loss 0.137056 acc 94.90% test: loss 0.355697 acc 89.22%
Epoch 15: train: loss 0.125393 acc 95.45% test: loss 0.296477 acc 90.62%
Epoch 16: train: loss 0.116349 acc 95.87% test: loss 0.300431 acc 91.68%
Epoch 17: train: loss 0.102414 acc 96.30% test: loss 0.303691 acc 90.73%
Epoch 18: train: loss 0.093724 acc 96.50% test: loss 0.308932 acc 91.12%
Epoch 19: train: loss 0.0911197 acc 96.69% test: loss 0.332449 acc 90.78%
```